

Objects Classification

✓ Problem Statement

✓ Business Context

In recent years, AI has been prominently used for classifying data as it is able to identify minute features which can not be observed with the naked eye. We have decided to test the accuracy of an AI model to identify images while trained with the regular data. Hence, images of everyday objects have been collected in the original form and provided to the model which will be trained.

After the model has been trained, we will pass images of the originals but have been altered in terms of brightness, blurriness, and contrast. From this we will be measuring how accurately the model will be able to classify this object in varying conditions.

By understanding the accuracy, this can be an understanding to how AI engineers in the world train their AI models to adapt to various conditions. For instance, an iPhone can identify a person's face even when the person is in a dark setting or just looks to the other side while holding their phone in front of them. Through this experiment, one will gain insight into how this phenomena works.

✓ Objective

Develop an image classification model to classify everyday objects based on various conditions such as brightness.

✓ Data Description

- Due to the volume of data, the images were converted to the object_images.npy file and the labels are also placed objects original.csv.csv, allowing one to work on the data readily.
- The dataset comprises of 10 everyday objects.

List of Objects

- Shoe
- Lead pencil
- Plastic spoon
- iPad
- Pen
- Eyewear
- Plant
- Bottle
- Book
- Teacup

Note: Please use GPU runtime to execute the code efficiently

✓ Installing and Importing the necessary libraries

```
!pip install numpy==1.25.2 pandas==2.0.3 seaborn==0.13.1 tensorflow==
```

[Show hidden output](#)

Note:

- After running the above cell, kindly restart the notebook kernel (for Jupyter Notebook) or runtime (for Google Colab) and run all cells sequentially from the next cell.
 - On executing the above line of code, you might see a warning regarding package dependencies. This error message can be ignored as the above code ensures that all necessary libraries and their dependencies are maintained to successfully execute the code in this notebook.
-

```

import os
import numpy as np # Importing numpy for Matrix Operations
import pandas as pd # Importing pandas to read CSV files
import matplotlib.pyplot as plt # Importing matplotlib for Plotting
import math # Importing math module to perform mathematical operations
import cv2 # Importing openCV for image processing
import seaborn as sns # Importing seaborn to plot graphs

# Tensorflow modules
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout,Flatten,Conv2D,MaxPooling2D
from tensorflow.keras.optimizers import Adam,SGD
from tensorflow.keras.models import Model
from keras.applications.vgg16 import VGG16

#Sklearn modules
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

# Display images using OpenCV
from google.colab.patches import cv2_imshow

#Imports functions for evaluating the performance of machine learning
from sklearn.metrics import confusion_matrix, f1_score,accuracy_score
from sklearn.metrics import mean_squared_error as mse

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')

```

```

# Set the seed using keras.utils.set_random_seed. This will set:
# 1) `numpy` seed
# 2) backend random seed
# 3) `python` random seed
tf.keras.utils.set_random_seed(812)

# If using TensorFlow, this will make GPU ops as deterministic as possible
# but it will affect the overall performance, so be mindful of that.
tf.config.experimental.enable_op_determinism()

```

✓ Loading the Data

```
# Uncomment and run the following code in case Google Colab is being used
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount('/content/drive', force_remount=True).

```
# Load the image file of the dataset
images = np.load('/content/drive/MyDrive/AIML Course/Case Studies/Competition/1000_images.npy')

# Load the labels file of the dataset
labels = pd.read_csv('/content/drive/MyDrive/AIML Course/Case Studies/Competition/1000_labels.csv')
```

✓ Data Overview

Let's print the shape of the images and labels

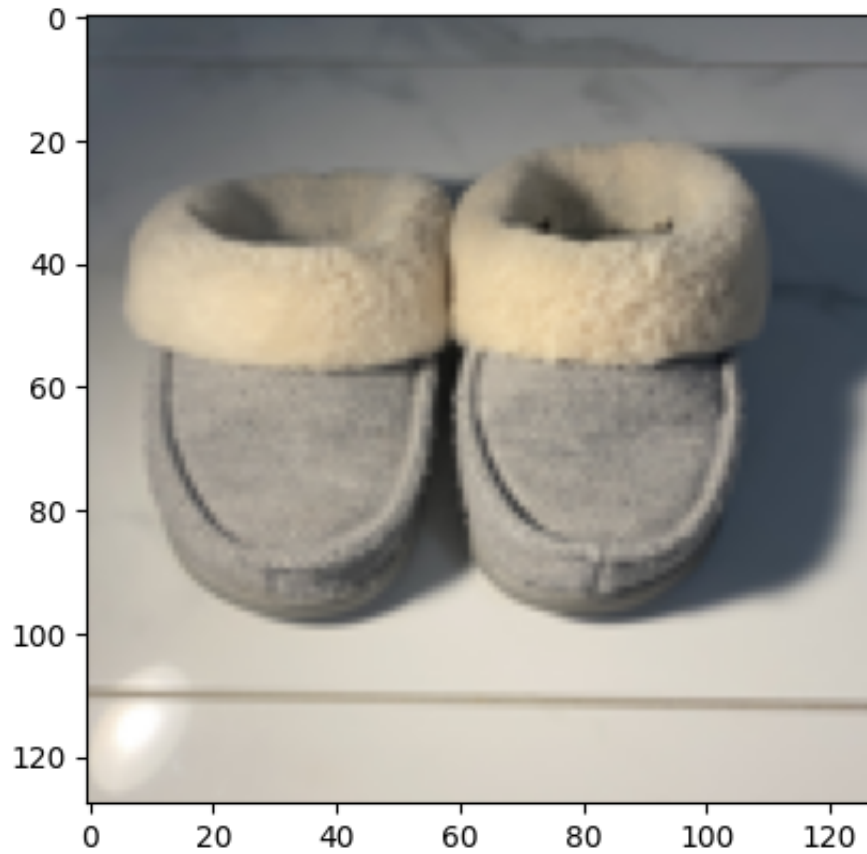
```
print(images.shape)
print(labels.shape)
```

```
(60, 128, 128, 3)
(60, 1)
```

There are 20 RGB images of shape 128 x 128 X 3, each image having 1 channel.

```
plt.imshow(images[0])
```

```
<matplotlib.image.AxesImage at 0x7c6af0482210>
```

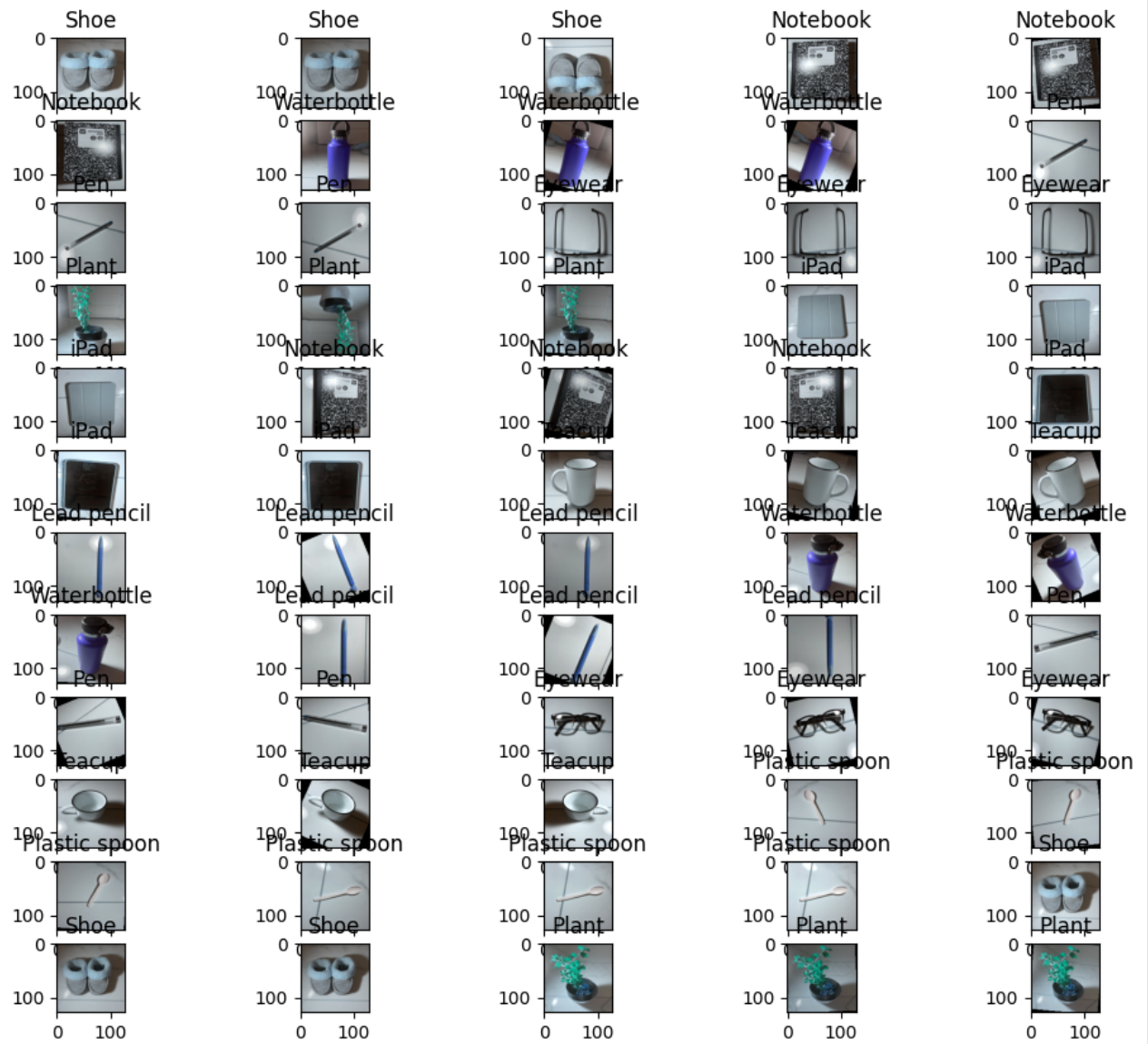


✓ Exploratory Data Analysis

```
# Converting the images from BGR to RGB using cvtColor function of Opencv
for i in range(len(images)):
    images[i] = cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB)
```

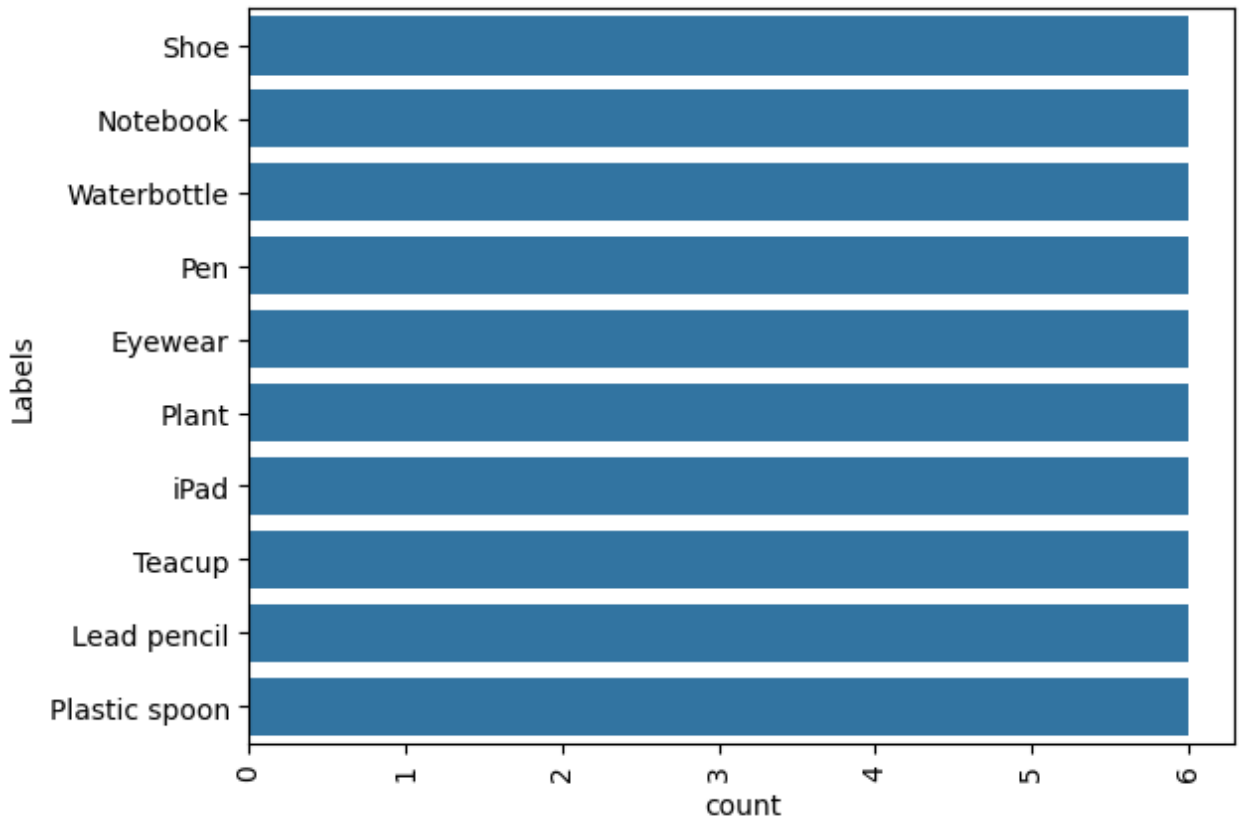
```
def plot_images(images, labels):
    num_classes=6
    unique_labels=np.unique(labels)
    keys=dict(labels['Labels'])
    rows = 12
    cols = 5
    fig = plt.figure(figsize=(12, 10))
    random_index=0
    for i in range(cols):
        for j in range(rows):
            #random_index = np.random.randint(0, len(labels))
            ax = fig.add_subplot(rows, cols, i * rows + j + 1)
            ax.imshow(images[random_index, :])
            ax.set_title(keys[random_index])
            random_index+=1
    plt.show()
```

plot_images(images, labels)



✓ Checking for data imbalance

```
sns.countplot(labels['Labels'])  
plt.xticks(rotation='vertical');
```



- As you can see from the above plot, the dataset is quite equal for all the objects.
- There are about 6 images in each category

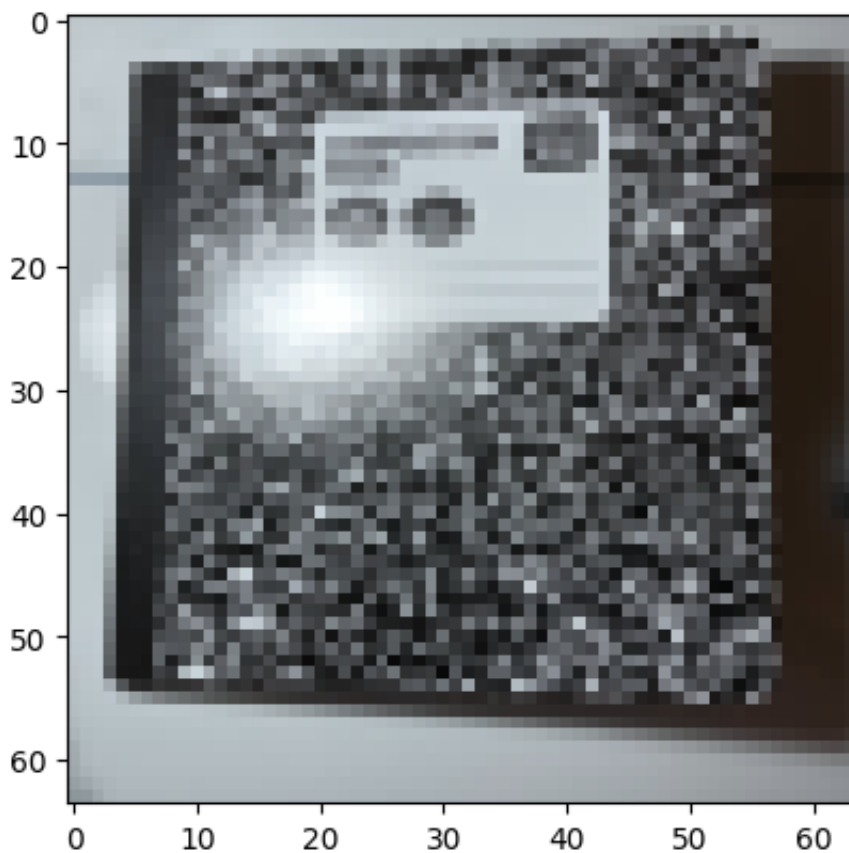
✓ Data Preprocessing

✓ Resizing images

As the size of the images is large, it may be computationally expensive to train on these larger images; therefore, it is preferable to reduce the image size from 128 to 64.

```
images_decreased = []
height = 64
width = 64
dimensions = (width, height)
for i in range(len(images)):
    images_decreased.append( cv2.resize(images[i], dimensions, interpola
```

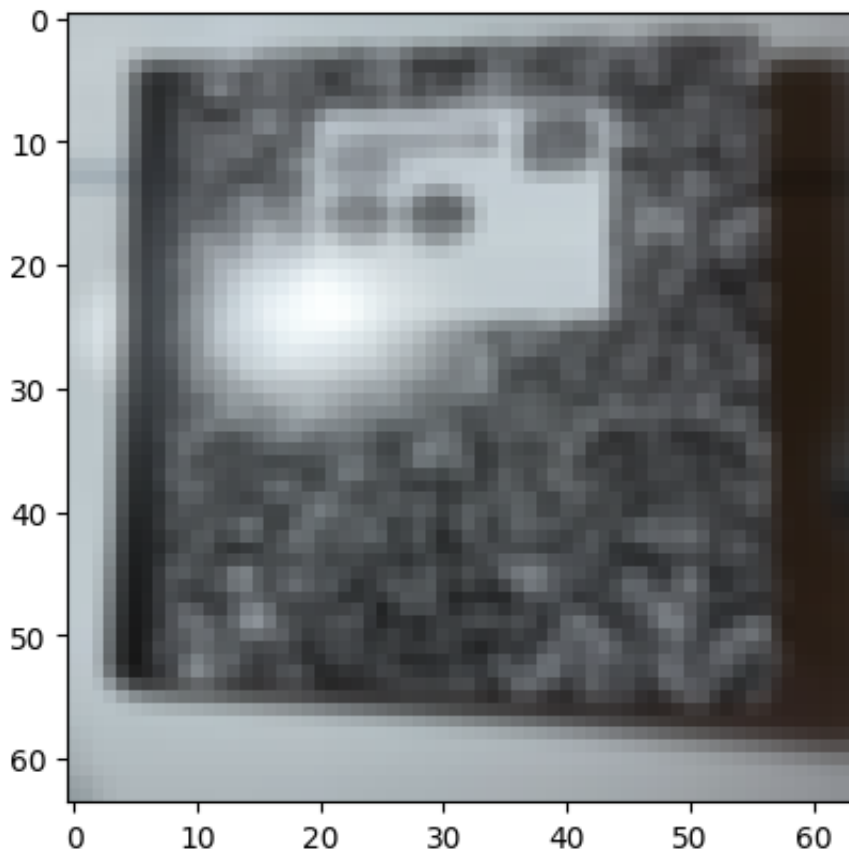
```
plt.imshow(images_decreased[3]);
```



✓ Visualizing images using Gaussian Blur

```
# Applying Gaussian Blur to denoise the images
images_gb=[]
for i in range(len(images_decreased)):
    images_gb.append(cv2.GaussianBlur(images_decreased[i], ksize =(3,3
```

```
plt.imshow(images_gb[3]);
```



- It appears that GaussianBlur would be ineffective because the blurred or denoised image does not seem to contain any relevant information, and the model would struggle to categorize these blurred images.

✓ Splitting the dataset

- As we have less images in our dataset, we will only use 10% of our data for testing, 10% of our data for validation and 80% of our data for training.
- We are using the `train_test_split()` function from scikit-learn. Here, we split the dataset into three parts, train, test and validation.

```
X_train, X_temp, y_train, y_temp = train_test_split(np.array(images_d), y, test_size=0.1)  
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.1)
```

```
print(X_train.shape,y_train.shape)
print(X_val.shape,y_val.shape)
print(X_test.shape,y_test.shape)
```

```
(54, 64, 64, 3) (54, 1)
(4, 64, 64, 3) (4, 1)
(2, 64, 64, 3) (2, 1)
```

✓ Encoding the target labels

- Convert labels from names to one hot vectors

```
# Convert labels from names to one hot vectors.
# We have already used encoding methods like onehotencoder and labeler
# Labelbinarizer works similar to onehotencoder

from sklearn.preprocessing import LabelBinarizer
enc = LabelBinarizer()
y_train_encoded = enc.fit_transform(y_train)
y_val_encoded=enc.transform(y_val)
y_test_encoded=enc.transform(y_test)
```

✓ Data Normalization

Since the **image pixel values range from 0-255**, our method of normalization here will be **scaling** - we shall **divide all the pixel values by 255 to standardize the images to have values between 0-1**.

```
# Normalizing the image pixels
X_train_normalized = X_train.astype('float32')/255.0
X_val_normalized = X_val.astype('float32')/255.0
X_test_normalized = X_test.astype('float32')/255.0
```

✓ Utility Functions

```

# defining a function to compute different metrics to check performance
def model_performance_classification(model, predictors, target):
    """
    Function to compute different metrics to check classification model performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    pred = model.predict(predictors).argmax(axis=1)

    target = target.argmax(axis=1)

    acc = accuracy_score(target, pred) # to compute Accuracy
    recall = recall_score(target, pred, average='weighted') # to compute Recall
    precision = precision_score(target, pred, average='weighted') # to compute Precision
    f1 = f1_score(target, pred, average='weighted') # to compute F1-score

    # creating a dataframe of metrics
    df_perf = pd.DataFrame({"Accuracy": acc, "Recall": recall, "Precision": precision, "F1-score": f1})

    return df_perf

```

```
def plot_confusion_matrix(model, predictors, target):
    """
    Function to plot the confusion matrix

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    pred = model.predict(predictors).argmax(axis=1)

    target = target.argmax(axis=1)

    # Plotting the Confusion Matrix using confusion matrix() function
    confusion_matrix = tf.math.confusion_matrix(target, pred)
    f, ax = plt.subplots(figsize=(10, 8))
    sns.heatmap(
        confusion_matrix,
        annot=True,
        linewidths=.4,
        fmt="d",
        square=True,
        ax=ax
    )
    plt.show()
```

✓ Model Building

✓ Model 1 (VGG-16 (Base))

- We will be loading a pre-built architecture - **VGG16**, which was trained on the ImageNet dataset and is the runner-up in the ImageNet competition in 2014.
- For training VGG16, we will directly use the convolutional and pooling layers and freeze their weights i.e. no training will be done on them. For classification, we will add a Flatten and a single dense layer.

```
vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(64, 64, 3))
vgg_model.summary()
```

Model: "vgg16"

Layer (type)	Output Shape	Param
input_layer_8 (InputLayer)	(None, 64, 64, 3)	
block1_conv1 (Conv2D)	(None, 64, 64, 64)	1,280
block1_conv2 (Conv2D)	(None, 64, 64, 64)	36,800
block1_pool (MaxPooling2D)	(None, 32, 32, 64)	
block2_conv1 (Conv2D)	(None, 32, 32, 128)	73,472
block2_conv2 (Conv2D)	(None, 32, 32, 128)	147,328
block2_pool (MaxPooling2D)	(None, 16, 16, 128)	
block3_conv1 (Conv2D)	(None, 16, 16, 256)	295,424
block3_conv2 (Conv2D)	(None, 16, 16, 256)	590,848
block3_conv3 (Conv2D)	(None, 16, 16, 256)	590,848
block3_pool (MaxPooling2D)	(None, 8, 8, 256)	
block4_conv1 (Conv2D)	(None, 8, 8, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 8, 8, 512)	2,359,424
block4_conv3 (Conv2D)	(None, 8, 8, 512)	2,359,424
block4_pool (MaxPooling2D)	(None, 4, 4, 512)	
block5_conv1 (Conv2D)	(None, 4, 4, 512)	2,359,424
block5_conv2 (Conv2D)	(None, 4, 4, 512)	2,359,424
block5_conv3 (Conv2D)	(None, 4, 4, 512)	2,359,424
block5_pool (MaxPooling2D)	(None, 2, 2, 512)	

Total params: 14,714,688 (56.13 MB)

Trainable params: 14,714,688 (56.13 MB)

Non-trainable params: 0 (0.00 B)

```
# Making all the layers of the VGG model non-trainable. i.e. freezing
for layer in vgg_model.layers:
    layer.trainable = False
```

```

model_1 = Sequential()

# Adding the convolutional part of the VGG16 model from above
model_1.add(vgg_model)

# Flattening the output of the VGG16 model because it is from a convo
model_1.add(Flatten())

# Adding a dense output layer
model_1.add(Dense(10, activation='softmax'))

```

```

opt=Adam()
# Compile model
model_1.compile(optimizer=opt, loss='categorical_crossentropy', metric

```

```

# Generating the summary of the model
model_1.summary()

```

Model: "sequential_6"

Layer (type)	Output Shape	Param
vgg16 (Functional)	(None, 2, 2, 512)	14,714
flatten_6 (Flatten)	(None, 2048)	
dense_14 (Dense)	(None, 10)	20

Total params: 14,735,178 (56.21 MB)
Trainable params: 20,490 (80.04 KB)
Non-trainable params: 14,714,688 (56.13 MB)

```

train_datagen = ImageDataGenerator()

```

```

# Epochs
epochs = 20
# Batch size
batch_size = 128

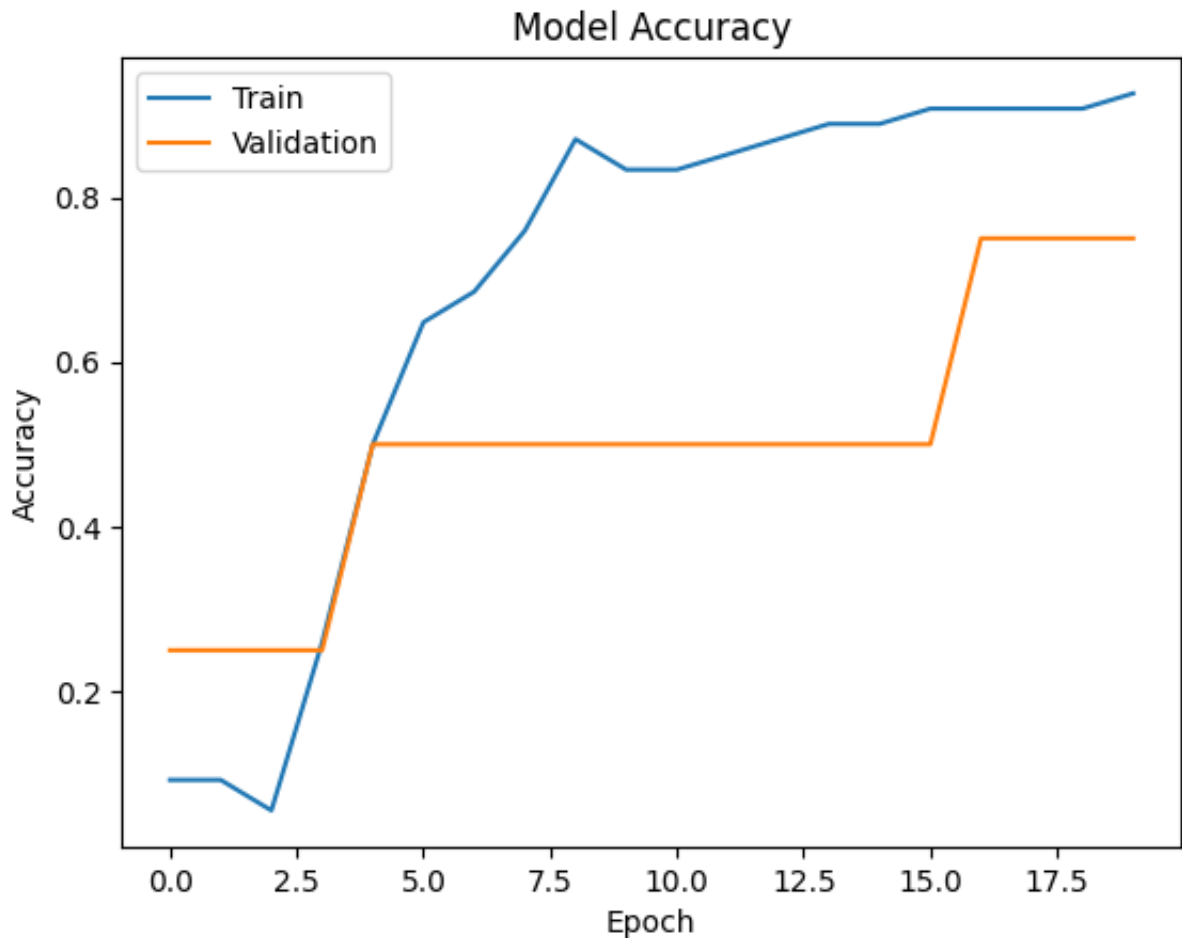
history_vgg16 = model_1.fit(train_datagen.flow(X_train_normalized,y_t
                                batch_size=batch_size,
                                seed=42,
                                shuffle=False),
                                epochs=epochs,
                                steps_per_epoch=X_train_normalized.shape[0] // ba
                                validation_data=(X_val_normalized,y_val_encoded),
                                verbose=1)

```

```
Epoch 1/20
1/1 _____ 2s 2s/step - accuracy: 0.0926 - loss: 2.8789 -
Epoch 2/20
1/1 _____ 0s 199ms/step - accuracy: 0.0926 - loss: 2.568
Epoch 3/20
1/1 _____ 0s 319ms/step - accuracy: 0.0556 - loss: 2.317
Epoch 4/20
1/1 _____ 0s 207ms/step - accuracy: 0.2593 - loss: 2.101
Epoch 5/20
1/1 _____ 0s 277ms/step - accuracy: 0.5000 - loss: 1.928
Epoch 6/20
1/1 _____ 0s 397ms/step - accuracy: 0.6481 - loss: 1.774
Epoch 7/20
1/1 _____ 0s 176ms/step - accuracy: 0.6852 - loss: 1.634
Epoch 8/20
1/1 _____ 0s 173ms/step - accuracy: 0.7593 - loss: 1.500
Epoch 9/20
1/1 _____ 0s 355ms/step - accuracy: 0.8704 - loss: 1.388
Epoch 10/20
1/1 _____ 0s 186ms/step - accuracy: 0.8333 - loss: 1.281
Epoch 11/20
1/1 _____ 0s 191ms/step - accuracy: 0.8333 - loss: 1.181
Epoch 12/20
1/1 _____ 0s 197ms/step - accuracy: 0.8519 - loss: 1.091
Epoch 13/20
1/1 _____ 0s 203ms/step - accuracy: 0.8704 - loss: 1.011
Epoch 14/20
1/1 _____ 0s 233ms/step - accuracy: 0.8889 - loss: 0.938
Epoch 15/20
1/1 _____ 0s 364ms/step - accuracy: 0.8889 - loss: 0.860
Epoch 16/20
1/1 _____ 0s 198ms/step - accuracy: 0.9074 - loss: 0.799
Epoch 17/20
1/1 _____ 0s 229ms/step - accuracy: 0.9074 - loss: 0.739
Epoch 18/20
1/1 _____ 0s 294ms/step - accuracy: 0.9074 - loss: 0.674
Epoch 19/20
1/1 _____ 0s 220ms/step - accuracy: 0.9074 - loss: 0.618
Epoch 20/20
1/1 _____ 0s 172ms/step - accuracy: 0.9259 - loss: 0.561
```



```
plt.plot(history_vgg16.history['accuracy'])
plt.plot(history_vgg16.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```



```
model_1_train_perf = model_performance_classification(model_1, X_train, y_train)

print("Train performance metrics")
print(model_1_train_perf)
```

WARNING:tensorflow:5 out of the last 14 calls to <function TensorFlowTrainer.train at 0x0000000000000000> will trigger a RuntimeWarning about potentially inefficient uses of tf.nn.softmax_cross_entropy_with_logits_v2. For more information, see https://www.tensorflow.org/api_guides/python/nn_ops#softmax_cross_entropy.

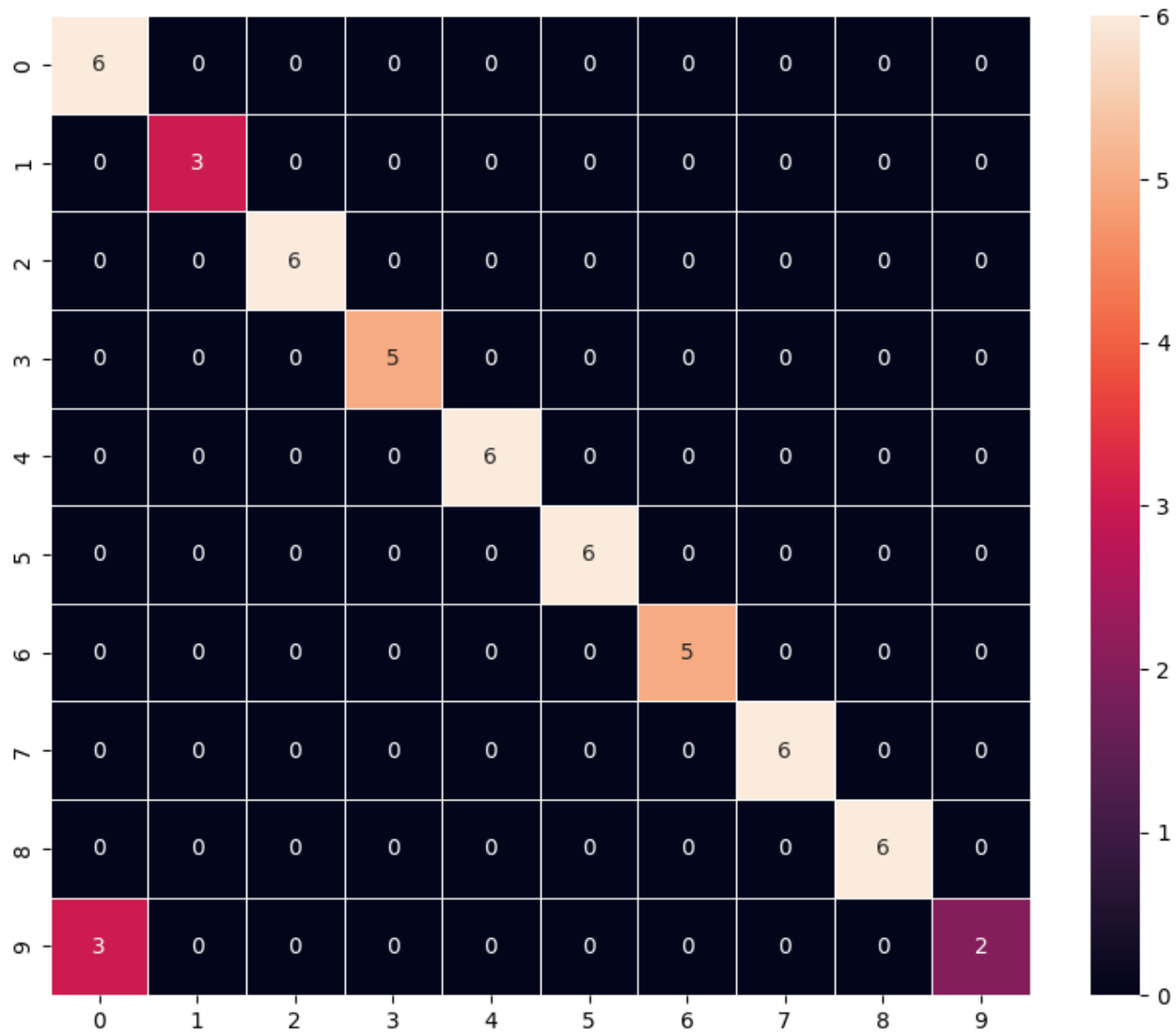
2/2 ————— 1s 242ms/step

Train performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.944444	0.944444	0.962963	0.938095

```
plot_confusion_matrix(model_1,X_train_normalized,y_train_encoded)
```

2/2 ————— 0s 32ms/step



```
model_1_valid_perf = model_performance_classification(model_1, X_val_1)

print("Validation performance metrics")
print(model_1_valid_perf)
```

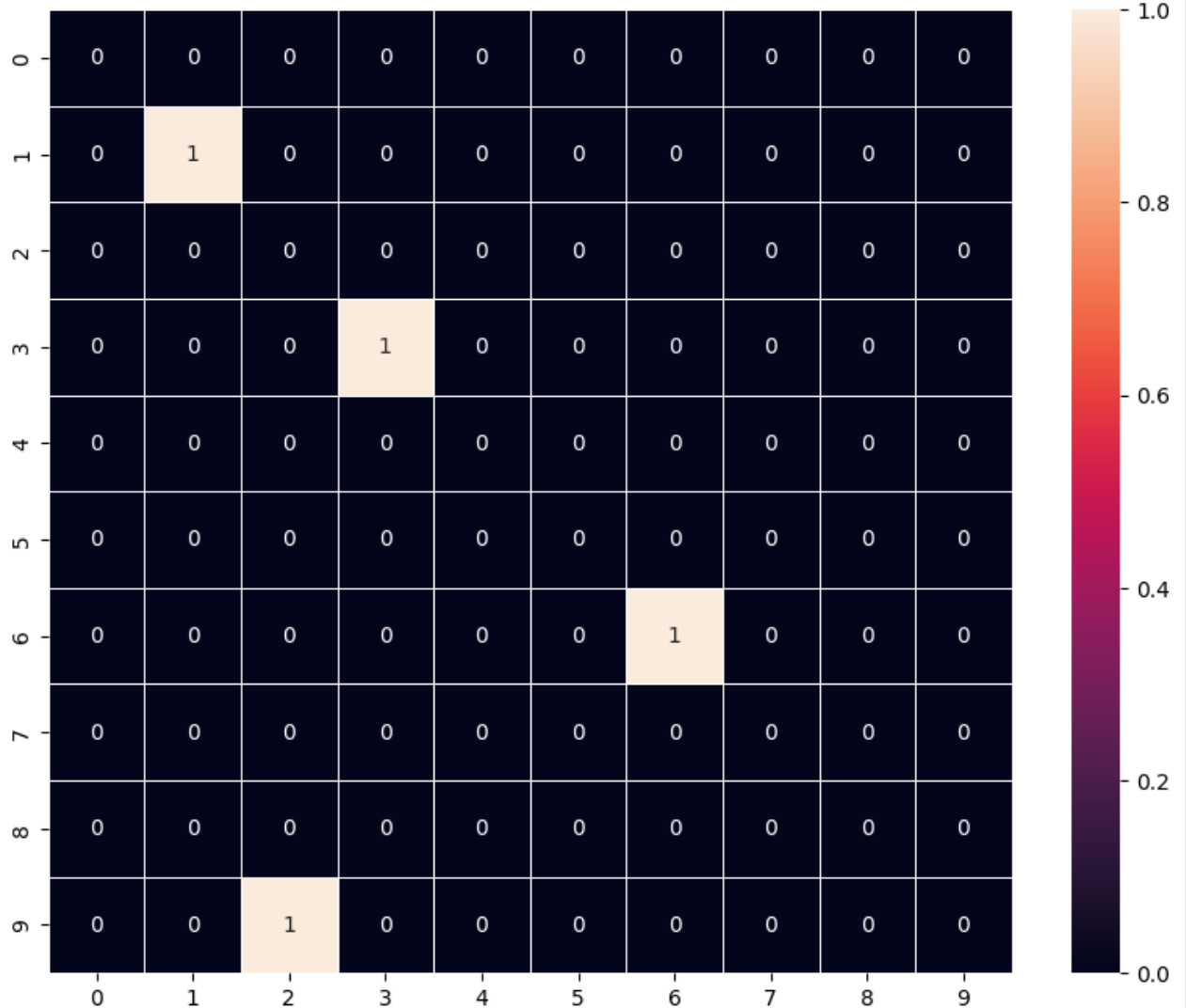
1/1  **0s** 45ms/step

Validation performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.75	0.75	0.75	0.75

```
plot_confusion_matrix(model_1,X_val_normalized,y_val_encoded)
```

1/1 ————— 0s 43ms/step



Visualizing the prediction:

```
# Visualizing the predicted and correct label of images from test data
plt.figure(figsize=(1,1))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_1.predict((X_val
```

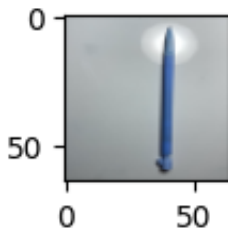
```

print('True Label', enc.inverse_transform(y_test_encoded)[1])

plt.figure(figsize=(1,1))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_1.predict((X_val_
print('True Label', enc.inverse_transform(y_test_encoded)[1])

plt.figure(figsize=(1,1))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_1.predict((X_val_
print('True Label', enc.inverse_transform(y_test_encoded)[1])

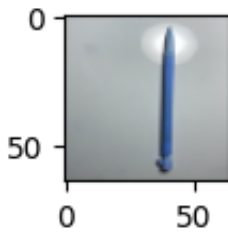
```



```

1/1 ————— 0s 43ms/step
Predicted Label ['Lead pencil']
True Label Lead pencil

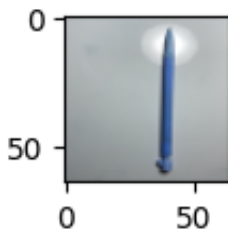
```



```

1/1 ————— 0s 44ms/step
Predicted Label ['Lead pencil']
True Label Lead pencil

```



```

1/1 ————— 0s 43ms/step
Predicted Label ['Lead pencil']
True Label Lead pencil

```

✓ Model 2 (VGG-16 (Base + FFNN))

- We will directly use the convolutional and pooling layers and freeze their weights i.e. no training will be done on them. For classification, we will add a Flatten layer and a Feed Forward Neural Network.

```
model_2 = Sequential()

# Adding the convolutional part of the VGG16 model from above
model_2.add(vgg_model)

# Flattening the output of the VGG16 model because it is from a convo
model_2.add(Flatten())

#Adding the Feed Forward neural network
model_2.add(Dense(256,activation='relu'))
model_2.add(Dropout(rate=0.4))
model_2.add(Dense(32,activation='relu'))

# Adding a dense output layer
model_2.add(Dense(10, activation='softmax'))
```

```
opt = Adam()
```

```
# Compile model
model_2.compile(optimizer=opt, loss='categorical_crossentropy', metric
```

```
# Generating the summary of the model
model_2.summary()
```

Model: "sequential_7"

Layer (type)	Output Shape	Param
vgg16 (Functional)	(None , 2, 2, 512)	14,714
flatten_7 (Flatten)	(None , 2048)	
dense_15 (Dense)	(None , 256)	524
dropout_4 (Dropout)	(None , 256)	
dense_16 (Dense)	(None , 32)	8
dense_17 (Dense)	(None , 10)	

Total params: 15,247,786 (58.17 MB)

Trainable params: 533,098 (2.03 MB)

Non-trainable params: 14,714,688 (56.13 MB)

```

history_vgg16 = model_2.fit(train_datagen.flow(X_train_normalized,y_train_normalized,
                                                batch_size=batch_size,
                                                seed=42,
                                                shuffle=False),
                             epochs=epochs,
                             steps_per_epoch=X_train_normalized.shape[0] // batch_size,
                             validation_data=(X_val_normalized,y_val_encoded),
                             verbose=1)

```

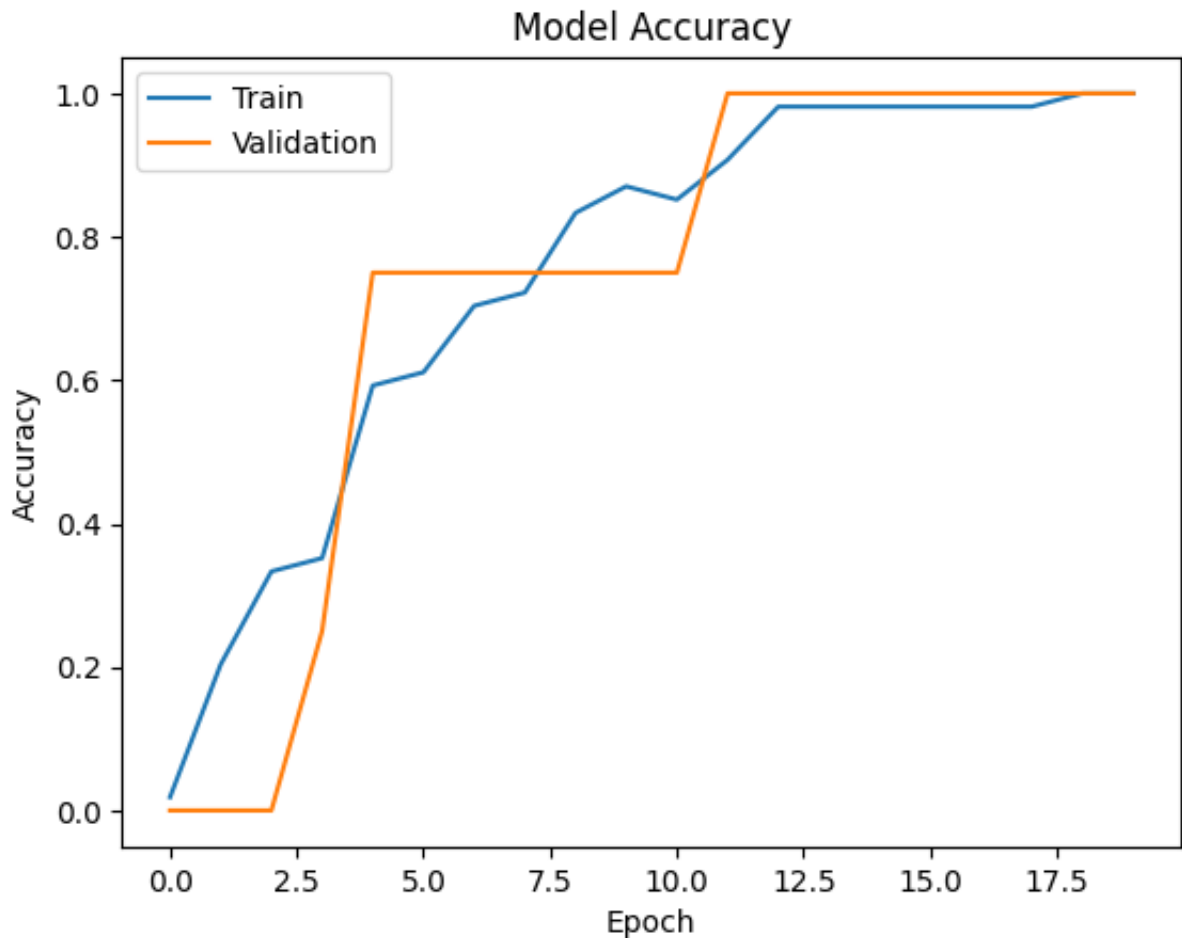
```

Epoch 1/20
1/1 _____ 2s 2s/step - accuracy: 0.0185 - loss: 2.7872
Epoch 2/20
1/1 _____ 0s 96ms/step - accuracy: 0.2037 - loss: 2.2840
Epoch 3/20
1/1 _____ 0s 103ms/step - accuracy: 0.3333 - loss: 2.0240
Epoch 4/20
1/1 _____ 0s 129ms/step - accuracy: 0.3519 - loss: 1.7872
Epoch 5/20
1/1 _____ 0s 95ms/step - accuracy: 0.5926 - loss: 1.6240
Epoch 6/20
1/1 _____ 0s 94ms/step - accuracy: 0.6111 - loss: 1.4370
Epoch 7/20
1/1 _____ 0s 132ms/step - accuracy: 0.7037 - loss: 1.2770
Epoch 8/20
1/1 _____ 0s 96ms/step - accuracy: 0.7222 - loss: 1.1420
Epoch 9/20
1/1 _____ 0s 96ms/step - accuracy: 0.8333 - loss: 0.8970
Epoch 10/20
1/1 _____ 0s 97ms/step - accuracy: 0.8704 - loss: 0.7530
Epoch 11/20
1/1 _____ 0s 94ms/step - accuracy: 0.8519 - loss: 0.7040
Epoch 12/20
1/1 _____ 0s 91ms/step - accuracy: 0.9074 - loss: 0.6500
Epoch 13/20
1/1 _____ 0s 97ms/step - accuracy: 0.9815 - loss: 0.4520
Epoch 14/20
1/1 _____ 0s 91ms/step - accuracy: 0.9815 - loss: 0.3450
Epoch 15/20
1/1 _____ 0s 96ms/step - accuracy: 0.9815 - loss: 0.3300
Epoch 16/20
1/1 _____ 0s 94ms/step - accuracy: 0.9815 - loss: 0.2610
Epoch 17/20
1/1 _____ 0s 93ms/step - accuracy: 0.9815 - loss: 0.1900
Epoch 18/20
1/1 _____ 0s 96ms/step - accuracy: 0.9815 - loss: 0.2230
Epoch 19/20
1/1 _____ 0s 101ms/step - accuracy: 1.0000 - loss: 0.1640
Epoch 20/20
1/1 _____ 0s 97ms/step - accuracy: 1.0000 - loss: 0.1260

```



```
plt.plot(history_vgg16.history['accuracy'])
plt.plot(history_vgg16.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```



```
model_2_train_perf = model_performance_classification(model_2, X_train, y_train)

print("Train performance metrics")
print(model_2_train_perf)
```

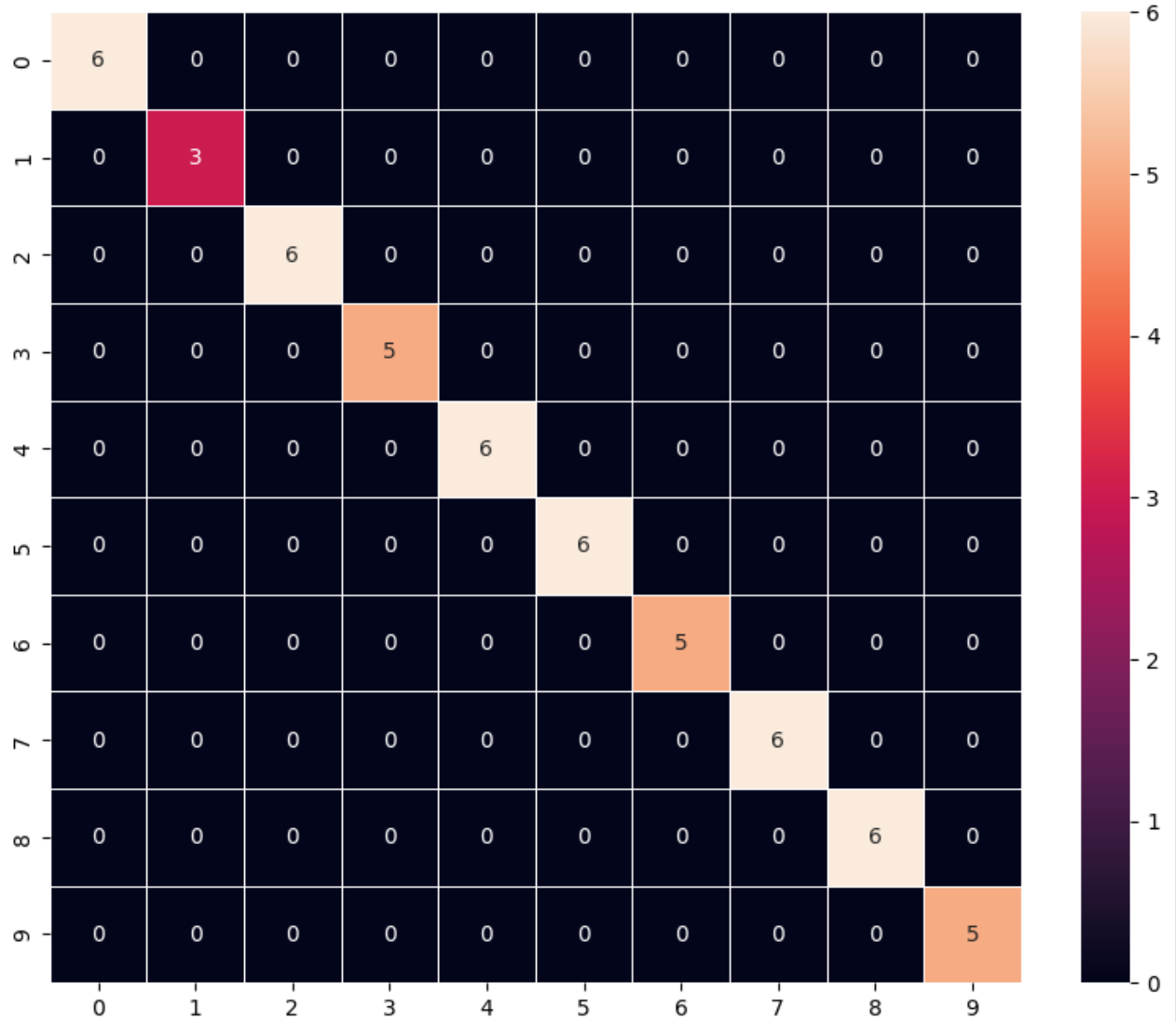
2/2 ————— 1s 289ms/step

Train performance metrics

	Accuracy	Recall	Precision	F1 Score
0	1.0	1.0	1.0	1.0

```
plot_confusion_matrix(model_2,X_train_normalized,y_train_encoded)
```

2/2 0s 37ms/step



```
model_2_valid_perf = model_performance_classification(model_2, X_val_1)

print("Validation performance metrics")
print(model_2_valid_perf)
```

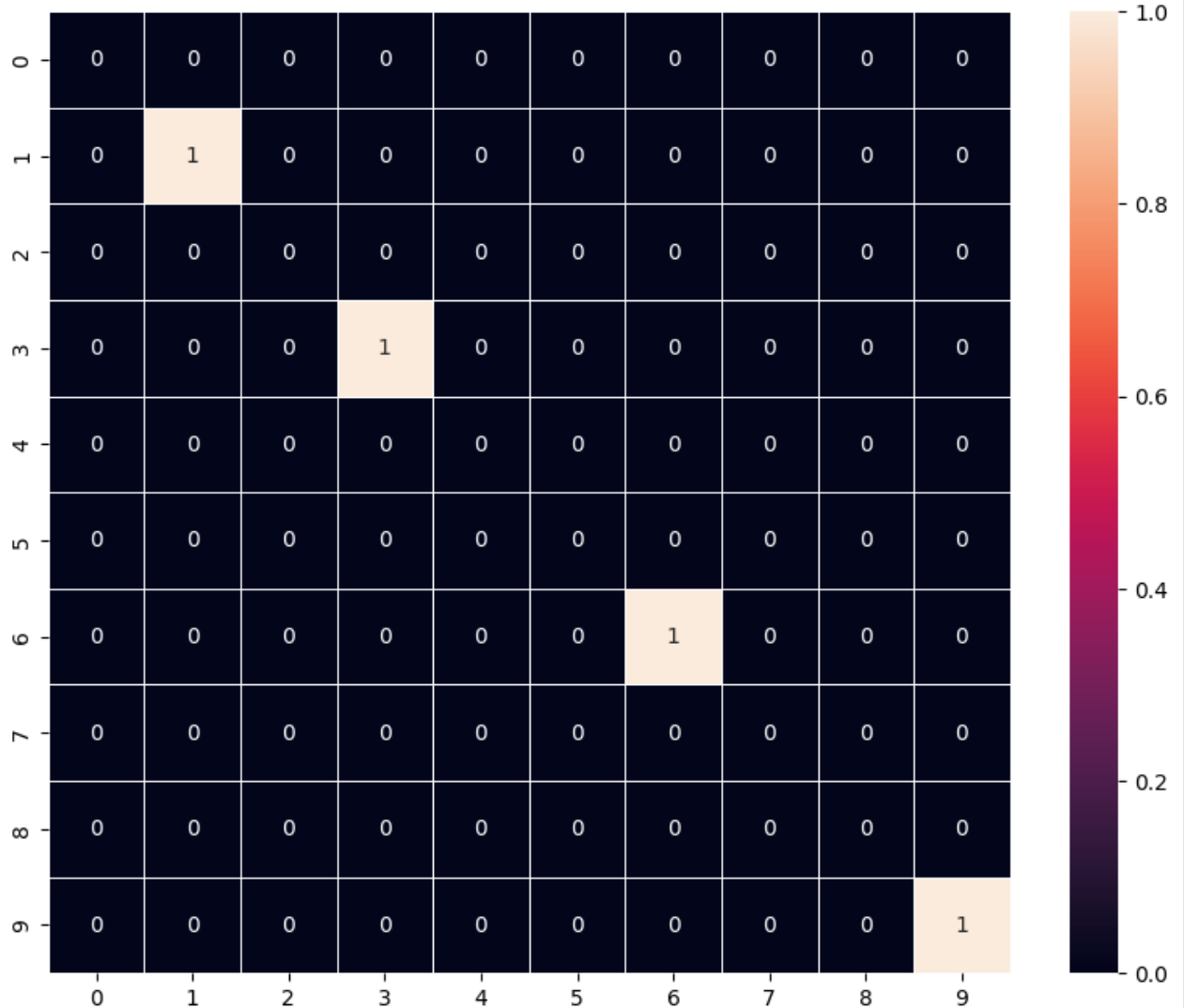
1/1  **0s** 65ms/step

Validation performance metrics

	Accuracy	Recall	Precision	F1 Score
0	1.0	1.0	1.0	1.0

```
plot_confusion_matrix(model_2,X_val_normalized,y_val_encoded)
```

1/1 ————— 0s 65ms/step



Visualizing the prediction:

```
# Visualizing the predicted and correct label of images from test data
plt.figure(figsize=(2,2))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_2.predict((X_val
```

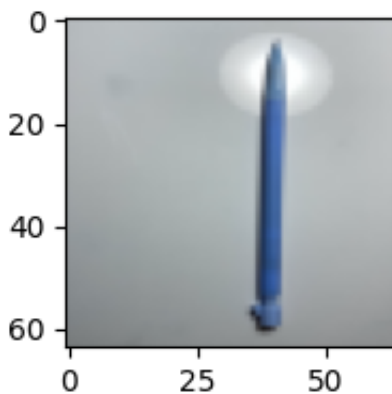
```

print('True Label', enc.inverse_transform(y_test_encoded)[1])

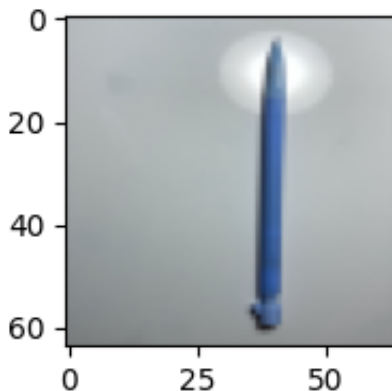
plt.figure(figsize=(2,2))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_2.predict((X_val_
print('True Label', enc.inverse_transform(y_test_encoded)[1])

plt.figure(figsize=(2,2))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_2.predict((X_val_
print('True Label', enc.inverse_transform(y_test_encoded)[1])

```

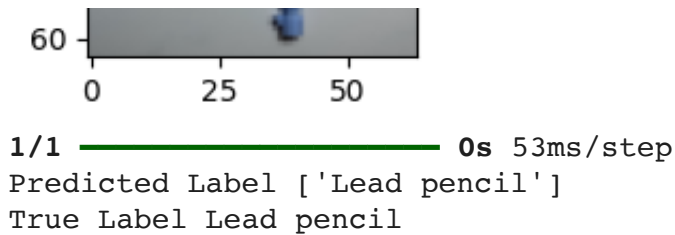


1/1 ————— 0s 56ms/step
 Predicted Label ['Lead pencil']
 True Label Lead pencil



1/1 ————— 0s 67ms/step
 Predicted Label ['Lead pencil']
 True Label Lead pencil





✓ Model 3 (VGG-16 (Base + FFNN + Data Augmentation))

- In most of the real-world case studies, it is challenging to acquire a large number of images and then train CNNs.
- To overcome this problem, one approach we might consider is **Data Augmentation**.
- CNNs have the property of **translational invariance**, which means they can recognise an object even if its appearance shifts translationally in some way. - Taking this attribute into account, we can augment the images using the techniques listed below
 - Horizontal Flip (should be set to True/False)
 - Vertical Flip (should be set to True/False)
 - Height Shift (should be between 0 and 1)
 - Width Shift (should be between 0 and 1)
 - Rotation (should be between 0 and 180)
 - Shear (should be between 0 and 1)
 - Zoom (should be between 0 and 1) etc.

Remember, **data augmentation should not be used in the validation/test data set.**

```

model_3 = Sequential()

# Adding the convolutional part of the VGG16 model from above
model_3.add(vgg_model)

# Flattening the output of the VGG16 model because it is from a convo
model_3.add(Flatten())

#Adding the Feed Forward neural network
model_3.add(Dense(256,activation='relu'))
model_3.add(Dropout(rate=0.4))
model_3.add(Dense(32,activation='relu'))

# Adding a dense output layer
model_3.add(Dense(10, activation='softmax'))

```

```

opt=Adam()
# Compile model
model_3.compile(optimizer=opt, loss='categorical_crossentropy', metric

```

```

# Generating the summary of the model
model_3.summary()

```

Model: "sequential_8"

Layer (type)	Output Shape	Param
vgg16 (Functional)	(None, 2, 2, 512)	14,714
flatten_8 (Flatten)	(None, 2048)	
dense_18 (Dense)	(None, 256)	524
dropout_5 (Dropout)	(None, 256)	
dense_19 (Dense)	(None, 32)	8
dense_20 (Dense)	(None, 10)	

Total params: 15,247,786 (58.17 MB)
Trainable params: 533,098 (2.03 MB)
Non-trainable params: 14,714,688 (56.13 MB)

```
# Applying data augmentation
train_datagen = ImageDataGenerator(
    rotation_range=20,
    fill_mode='nearest',width_shift_range=0.
)
```

```

history_vgg16 = model_3.fit(train_datagen.flow(X_train_normalized,y_train_normalized,
                                                batch_size=batch_size,
                                                seed=42,
                                                shuffle=False),
                            epochs=epochs,
                            steps_per_epoch=X_train_normalized.shape[0] // batch_size,
                            validation_data=(X_val_normalized,y_val_encoded),
                            verbose=1)

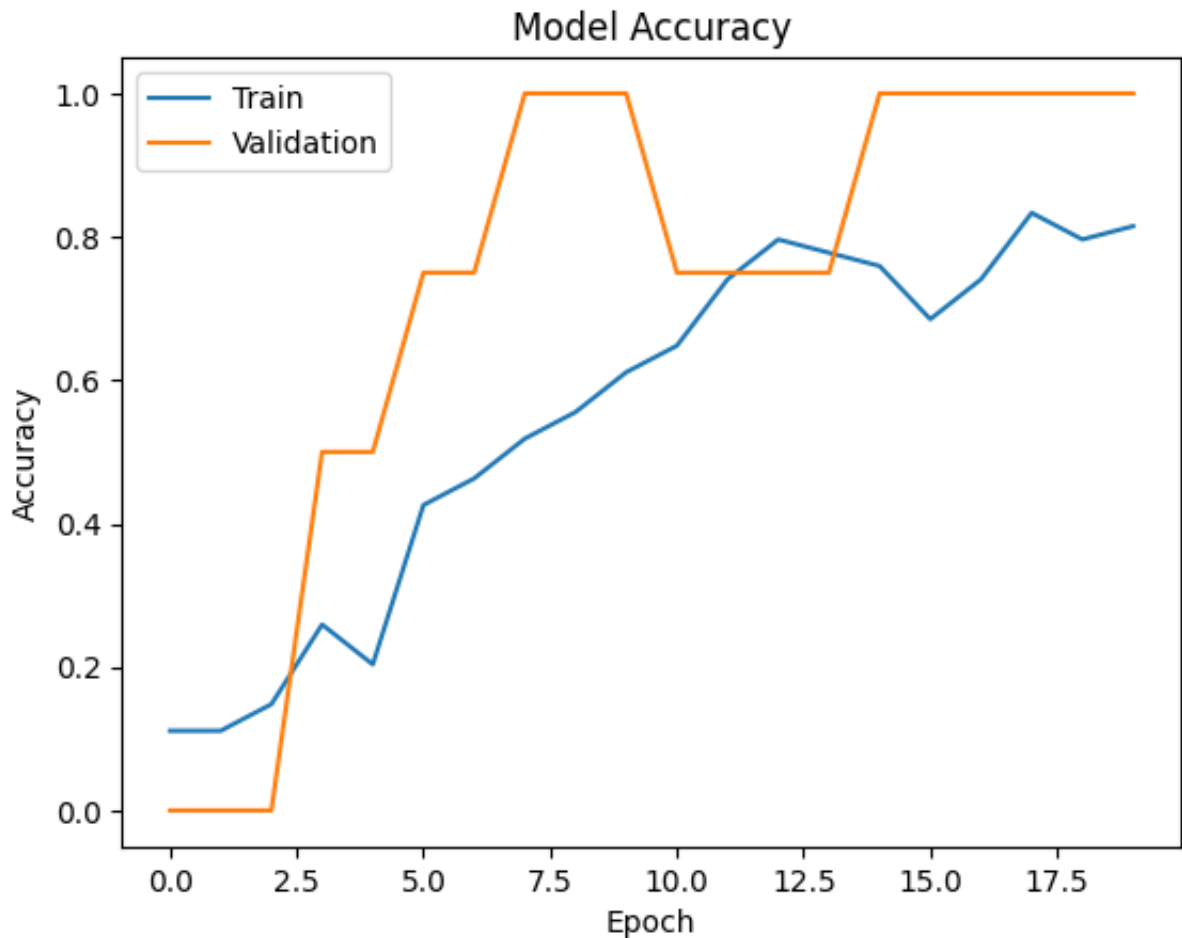
```

```

Epoch 1/20
1/1 _____ 4s 4s/step - accuracy: 0.1111 - loss: 2.6630
Epoch 2/20
1/1 _____ 0s 279ms/step - accuracy: 0.1111 - loss: 2.3400
Epoch 3/20
1/1 _____ 0s 420ms/step - accuracy: 0.1481 - loss: 2.1900
Epoch 4/20
1/1 _____ 0s 341ms/step - accuracy: 0.2593 - loss: 1.9800
Epoch 5/20
1/1 _____ 1s 633ms/step - accuracy: 0.2037 - loss: 2.0200
Epoch 6/20
1/1 _____ 1s 550ms/step - accuracy: 0.4259 - loss: 1.7900
Epoch 7/20
1/1 _____ 0s 384ms/step - accuracy: 0.4630 - loss: 1.6800
Epoch 8/20
1/1 _____ 0s 350ms/step - accuracy: 0.5185 - loss: 1.4800
Epoch 9/20
1/1 _____ 0s 325ms/step - accuracy: 0.5556 - loss: 1.4900
Epoch 10/20
1/1 _____ 1s 684ms/step - accuracy: 0.6111 - loss: 1.3900
Epoch 11/20
1/1 _____ 0s 452ms/step - accuracy: 0.6481 - loss: 1.2200
Epoch 12/20
1/1 _____ 0s 460ms/step - accuracy: 0.7407 - loss: 1.0900
Epoch 13/20
1/1 _____ 0s 313ms/step - accuracy: 0.7963 - loss: 0.9400
Epoch 14/20
1/1 _____ 0s 369ms/step - accuracy: 0.7778 - loss: 0.9000
Epoch 15/20
1/1 _____ 1s 700ms/step - accuracy: 0.7593 - loss: 0.7600
Epoch 16/20
1/1 _____ 1s 826ms/step - accuracy: 0.6852 - loss: 1.0300
Epoch 17/20
1/1 _____ 1s 673ms/step - accuracy: 0.7407 - loss: 0.7500
Epoch 18/20
1/1 _____ 1s 1s/step - accuracy: 0.8333 - loss: 0.5666
Epoch 19/20
1/1 _____ 1s 595ms/step - accuracy: 0.7963 - loss: 0.7000
Epoch 20/20
1/1 _____ 1s 628ms/step - accuracy: 0.8148 - loss: 0.7400

```

```
plt.plot(history_vgg16.history['accuracy'])
plt.plot(history_vgg16.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```



```
model_3_train_perf = model_performance_classification(model_3, X_train, y_train)

print("Train performance metrics")
print(model_3_train_perf)
```

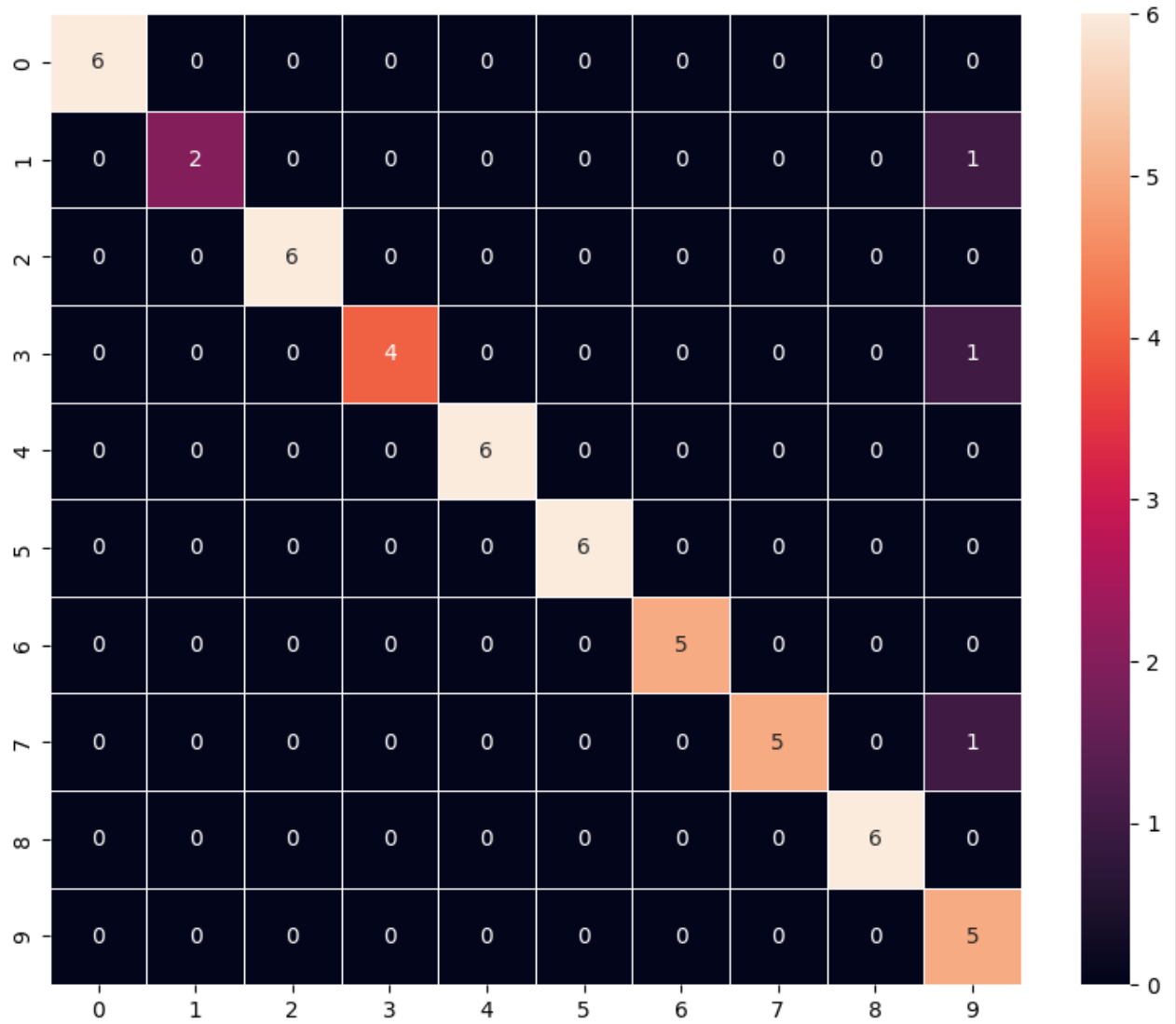
2/2 ————— 1s 395ms/step

Train performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.944444	0.944444	0.965278	0.947132

```
plot_confusion_matrix(model_3,X_train_normalized,y_train_encoded)
```

2/2 0s 51ms/step



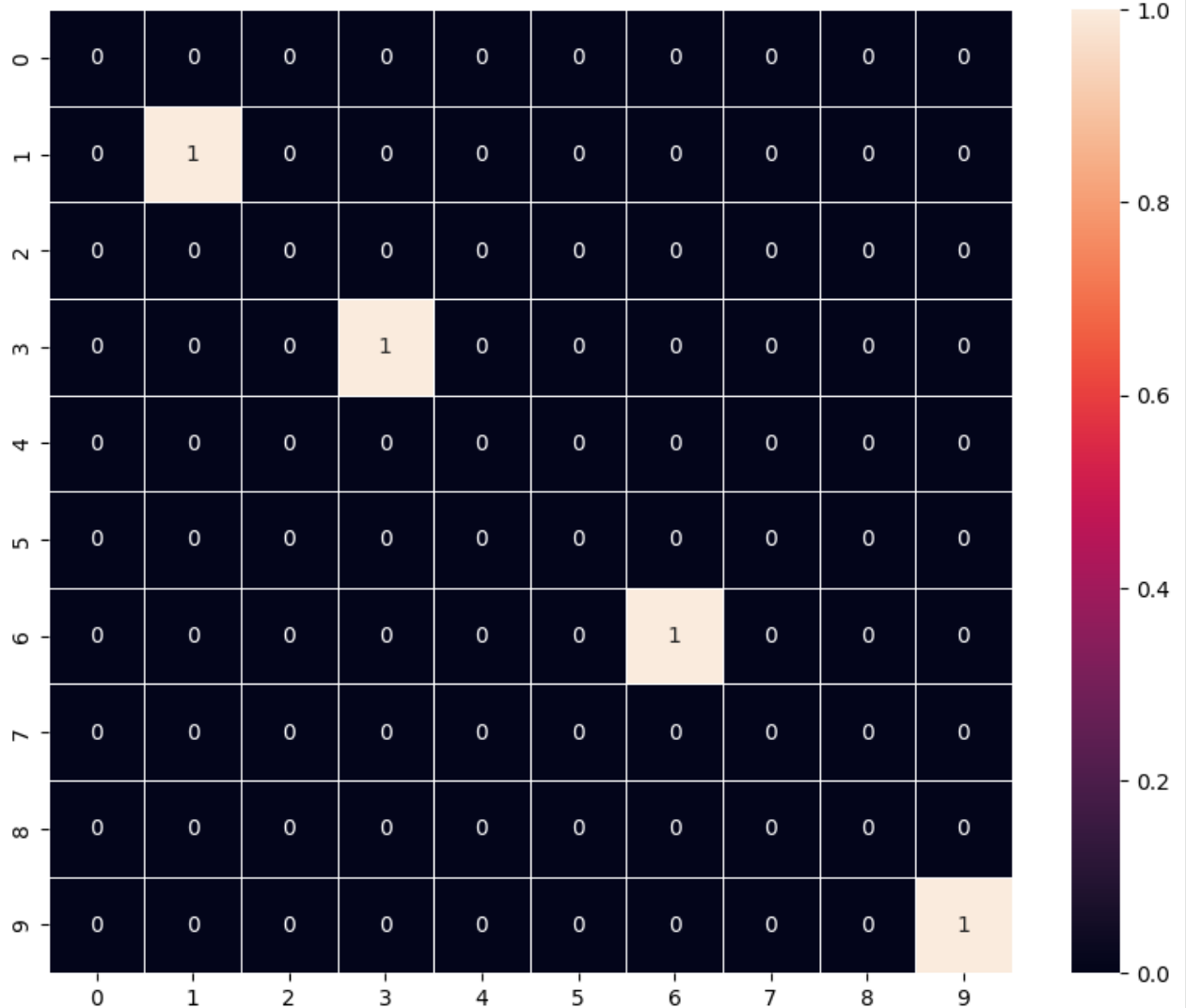
```
model_3_valid_perf = model_performance_classification(model_3, X_val_1)

print("Validation performance metrics")
print(model_3_valid_perf)
```

```
1/1  0s 141ms/step
Validation performance metrics
  Accuracy  Recall  Precision  F1 Score
0         1.0     1.0         1.0       1.0
```

```
plot_confusion_matrix(model_3,X_val_normalized,y_val_encoded)
```

1/1 ————— 0s 95ms/step



Visualizing the prediction:

```
# Visualizing the predicted and correct label of images from test data
plt.figure(figsize=(2,2))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_3.predict((X_val
```

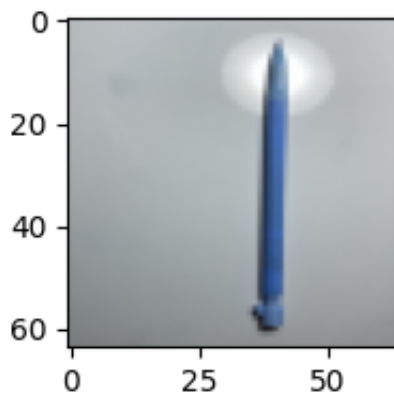
```

print('True Label', enc.inverse_transform(y_test_encoded)[1])

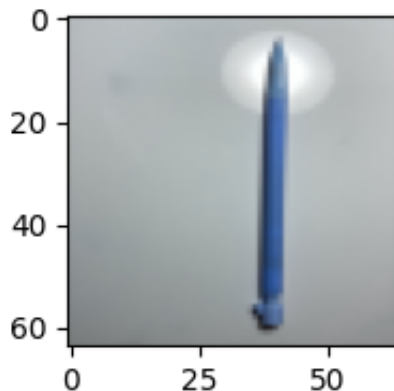
plt.figure(figsize=(2,2))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_3.predict((X_val_
print('True Label', enc.inverse_transform(y_test_encoded)[1])

plt.figure(figsize=(2,2))
plt.imshow(X_val[1])
plt.show()
print('Predicted Label', enc.inverse_transform(model_3.predict((X_val_
print('True Label', enc.inverse_transform(y_test_encoded)[1])

```

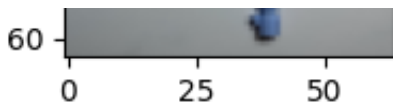


1/1 ————— 0s 110ms/step
 Predicted Label ['Lead pencil']
 True Label Lead pencil



1/1 ————— 0s 63ms/step
 Predicted Label ['Lead pencil']
 True Label Lead pencil





1/1 ————— 0s 75ms/step
Predicted Label ['Lead pencil']
True Label Lead pencil

Model Performance Comparison and Final Model Selection

```
# training performance comparison

models_train_comp_df = pd.concat(
    [
        model_1_train_perf.T,
        model_2_train_perf.T,
        model_3_train_perf.T,
    ],
    axis=1,
)
models_train_comp_df.columns = [
    "VGG-16 (Base)", "VGG-16 (Base+FFNN)", "VGG-16 (Base+FFNN+Data Aug)"
]
```

```
models_valid_comp_df = pd.concat(
    [
        model_1_valid_perf.T,
        model_2_valid_perf.T,
        model_3_valid_perf.T,
    ],
    axis=1,
)
models_valid_comp_df.columns = [
    "VGG-16 (Base)", "VGG-16 (Base+FFNN)", "VGG-16 (Base+FFNN+Data Aug)"
]
```

models_train_comp_df

	VGG-16 (Base)	VGG-16 (Base+FFNN)	VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.944444	1.0	0.944444
Recall	0.944444	1.0	0.944444
Precision	0.962963	1.0	0.965278
F1 Score	0.938095	1.0	0.947132

models_valid_comp_df

	VGG-16 (Base)	VGG-16 (Base+FFNN)	VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.75	1.0	1.0
Recall	0.75	1.0	1.0
Precision	0.75	1.0	1.0
F1 Score	0.75	1.0	1.0

models_train_comp_df - models_valid_comp_df

	VGG-16 (Base)	VGG-16 (Base+FFNN)	VGG-16 (Base+FFNN+Data Aug)
Accuracy	0.194444	0.0	-0.055556
Recall	0.194444	0.0	-0.055556
Precision	0.212963	0.0	-0.034722
F1 Score	0.188095	0.0	-0.052868

- VGG-16 (Base+FFNN) has achieved the highest performance in the training set. Also, it has achieved a decent performance of ~0.61 in the validation set.
- VGG-16(Base+FFNN+Data Aug) has achieved a least performance of ~0.62 in the training set.
-
- We'll move ahead with the VGG-16 (Base+FFNN) as our final model.

✓ Test Performance

```
model_2_test_perf = model_performance_classification(model_2, X_test_i
```

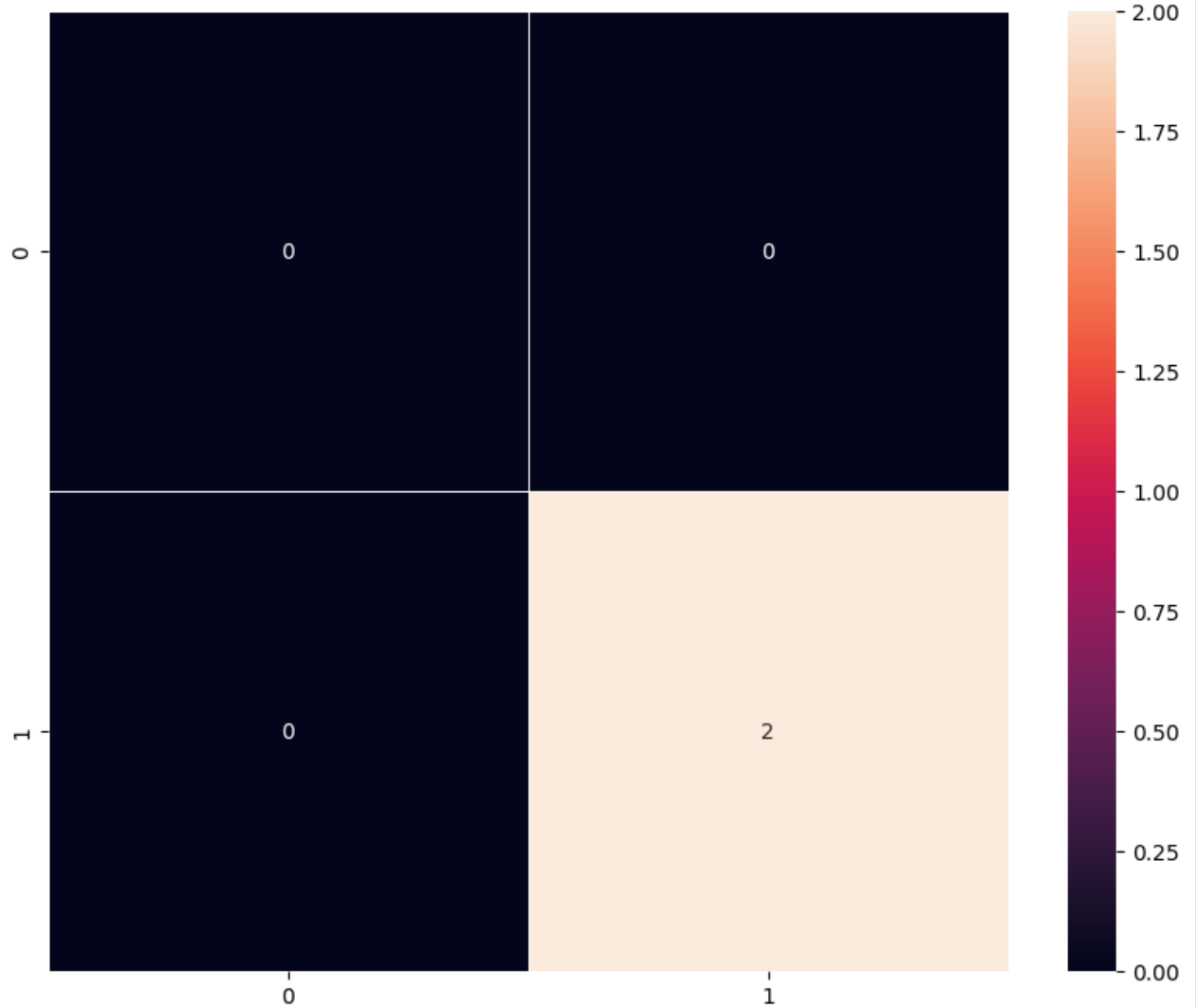
1/1  **0s** 106ms/step

```
model_2_test_perf
```

	Accuracy	Recall	Precision	F1 Score
0	1.0	1.0	1.0	1.0

```
plot_confusion_matrix(model_2, X_test_normalized, y_test_encoded)
```

1/1 ————— 0s 78ms/step



✓ Model Deployment

```
# Create a folder for storing the files needed for web app deployment  
os.makedirs("backend_files", exist_ok=True)
```

```
# Define the file path to save (serialize) the trained model along with
saved_model_path = "backend_files/learn_model.joblib"
```

```
!pip install joblib==1.4.2 requests==2.32.3 huggingface_hub==0.30.1 -
```

```
# To serialize the model
import joblib

# os related functionalities
import os

# API request
import requests

# for hugging face space authentication to upload files
from huggingface_hub import login, HfApi
```

```
# Save the best trained model pipeline
# Save only the weights of the model
kera_model_weights_filename = "learn_model_weights.weights.h5" # Define
# The full path inside the backend_files directory for saving
saved_keras_model_weights_full_path = os.path.join("backend_files", kera_model_weights_filename)
model_3.save_weights(saved_keras_model_weights_full_path)
```

```
print(f"Keras Model weights saved successfully at {saved_keras_model_weights_full_path}")
```

```
Keras Model weights saved successfully at backend_files/learn_model_weights.weights.h5
```

```
# Copy the labels CSV file to the backend_files directory
import shutil
import os

source_path = '/content/drive/MyDrive/AI/ML Course/Case Studies/Computer Vision/object_labels_augmented.csv'
destination_path = 'backend_files/object_labels_augmented.csv'

# Ensure the destination directory exists
os.makedirs(os.path.dirname(destination_path), exist_ok=True)

shutil.copy(source_path, destination_path)
print(f"Copied {source_path} to {destination_path}")
```

```
Copied /content/drive/MyDrive/AI/ML Course/Case Studies/Computer Vision/object_labels_augmented.csv to backend_files/object_labels_augmented.csv
```

```

%%writefile backend_files/app.py

import numpy as np
import tensorflow as tf
import pandas as pd
from flask import Flask, request, jsonify
import base64
import cv2
from sklearn.preprocessing import LabelBinarizer
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Dropout

# Initialize Flask app
learn_api = Flask("Objects Prediction")

# Re-create the model architecture exactly as it was trained
vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Making all the layers of the VGG model non-trainable, as they were pre-trained
for layer in vgg_model.layers:
    layer.trainable = False

model = Sequential()
model.add(vgg_model)
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(rate=0.4))
model.add(Dense(32, activation='relu'))
model.add(Dense(10, activation='softmax'))

# Load the weights into the re-created model
model.load_weights("learn_model_weights.weights.h5")

# Re-create and fit the LabelBinarizer
# Load the labels CSV from the local backend_files directory
all_unique_labels = sorted(pd.read_csv('object_labels_augmented.csv'))
enc = LabelBinarizer()
enc.fit(all_unique_labels)

# Define image dimensions for preprocessing
height = 64
width = 64
dimensions = (width, height)

# Define a route for the home page
@learn_api.get('/')

```

```

def home():
    return "Welcome to the Object Prediction System"

# Define an endpoint to predict object from an image
@learn_api.post('/v1/objectprediction')
def predict_object():
    data = request.get_json()
    image_b64 = data['image_file']
    image_bytes = base64.b64decode(image_b64)
    np_arr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)

    img_resized = cv2.resize(img, dimensions, interpolation=cv2.INTER_
    img_normalized = img_resized.astype('float32') / 255.0
    img_for_prediction = np.expand_dims(img_normalized, axis=0)

    prediction_proba = model.predict(img_for_prediction)
    predicted_class_index = np.argmax(prediction_proba)

    predicted_one_hot = np.zeros((1, len(enc.classes_)))
    predicted_one_hot[0, predicted_class_index] = 1
    predicted_label = enc.inverse_transform(predicted_one_hot)[0]

    return jsonify({'Predicted_Object': predicted_label})

if __name__ == '__main__':
    learn_api.run(debug=True, host='0.0.0.0', port=7860)

```

Overwriting backend_files/app.py

```

# Load the saved model pipeline from the file
saved_model = joblib.load("backend_files/learn_model.joblib")

# Confirm the model is loaded
print("Model loaded successfully.")

```

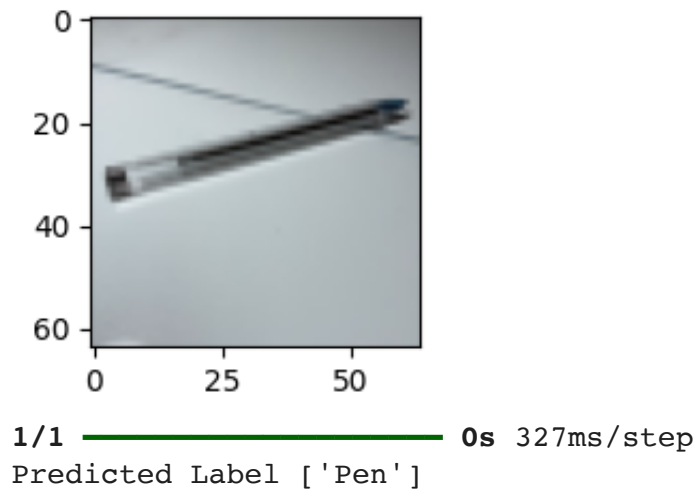
Model loaded successfully.

saved_model

<Sequential name=sequential_8, built=True>

```
#saved_model.predict(X_test)

plt.figure(figsize=(2,2))
plt.imshow(X_val[3])
plt.show()
print('Predicted Label', enc.inverse_transform(saved_model.predict((X_
#print('True Label', enc.inverse_transform(y_test_encoded)[2]))
```



✓ Deployment Backend

✓ Flask Web Framework

```
%%writefile backend_files/app.py

import numpy as np
import tensorflow as tf
import pandas as pd
from flask import Flask, request, jsonify
import base64
import cv2
from sklearn.preprocessing import LabelBinarizer
from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Dropout

# Initialize Flask app
learn_api = Flask("Objects Prediction")
```

```

# Re-create the model architecture exactly as it was trained
vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Making all the layers of the VGG model non-trainable, as they were (frozen)
for layer in vgg_model.layers:
    layer.trainable = False

model = Sequential()
model.add(vgg_model)
model.add(Flatten())
model.add(Dense(256, activation='relu'))
model.add(Dropout(rate=0.4))
model.add(Dense(32, activation='relu'))
model.add(Dense(10, activation='softmax'))

# Load the weights into the re-created model
model.load_weights("learn_model_weights.weights.h5")

# Re-create and fit the LabelBinarizer
all_unique_labels = sorted(pd.read_csv('/content/drive/MyDrive/AI/ML_Course/labels.csv')['class'].unique())
enc = LabelBinarizer()
enc.fit(all_unique_labels)

# Define image dimensions for preprocessing
height = 64
width = 64
dimensions = (width, height)

# Define a route for the home page
@learn_api.get('/')
def home():
    return "Welcome to the Object Prediction System"

# Define an endpoint to predict object from an image
@learn_api.post('/v1/objectprediction')
def predict_object():
    data = request.get_json()
    image_b64 = data['image_file']
    image_bytes = base64.b64decode(image_b64)
    np_arr = np.frombuffer(image_bytes, np.uint8)
    img = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)

    img_resized = cv2.resize(img, dimensions, interpolation=cv2.INTER_LINEAR)
    img_normalized = img_resized.astype('float32') / 255.0
    img_for_prediction = np.expand_dims(img_normalized, axis=0)

    prediction_proba = model.predict(img_for_prediction)
    predicted_class_index = np.argmax(prediction_proba)

```

```
predicted_one_hot = np.zeros((1, len(enc.classes_)))
predicted_one_hot[0, predicted_class_index] = 1
predicted_label = enc.inverse_transform(predicted_one_hot)[0]

return jsonify({'Predicted_Object': predicted_label})

if __name__ == '__main__':
    learn_api.run(debug=True, host='0.0.0.0', port=7860)
```

Overwriting backend_files/app.py


```

# Update the repo_id for the backend deployment to a dedicated space
backend_repo_id = "nkrishnappa/objectprediction" # Using the space you

# Try to create the repository for the Backend Hugging Face Space
try:
    create_repo(backend_repo_id.split('/')[1], # Extract just the repo name
                repo_type="space",
                space_sdk="docker",
                private=False
    )
except Exception as e:
    if "RepositoryAlreadyExistsError" in str(e):
        print(f"Repository {backend_repo_id} already exists. Skipping")
    else:
        print(f"Error creating repository: {e}")

# Upload backend files to the dedicated backend space
login(token=access_key)
api = HfApi()

print(f"Uploading backend files to {backend_repo_id}...")
api.upload_folder(
    folder_path="backend_files",
    repo_id=backend_repo_id,
    repo_type="space",
)
print(f"Backend files uploaded to {backend_repo_id}.")

# Update the frontend app.py to point to the new backend API URL
frontend_backend_api_url = f"https://{backend_repo_id.replace('/', '-')}.raw.githubusercontent.com"

```

```

Error creating repository: 409 Client Error: Conflict for url: https://huggingface.co/nkrishnappa/objectprediction

You already created this space repo: nkrishnappa/objectprediction
Uploading backend files to nkrishnappa/objectprediction...
Uploading...: 100% 196M/196M [00:02<00:00, 103MB/s]
Backend files uploaded to nkrishnappa/objectprediction.

```

✓ Dependencies File

```
%%writefile backend_files/requirements.txt
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.6.1
seaborn==0.13.2
joblib==1.4.2
xgboost==2.1.4
Werkzeug==2.2.2
flask==2.2.2
gunicorn==20.1.0
requests==2.32.3
uvicorn[standard]
streamlit==1.43.2
opencv-python==4.9.0.80
tensorflow==2.15.0
```

Overwriting backend_files/requirements.txt

✓ Dockerfile

```
%%writefile backend_files/Dockerfile
FROM python:3.9-bullseye

# Install necessary system dependencies for libGL.so.1
RUN apt-get update && apt-get install -y --no-install-recommends libgl1-mesa-glx

# Set the working directory inside the container
WORKDIR /app

# Copy all files from the current directory to the container's working directory
COPY . .

# Install dependencies from the requirements file without using cache
RUN pip install --no-cache-dir --upgrade -r requirements.txt

# Define the command to start the application using Gunicorn with 4 workers
# - `w 4`: Uses 4 worker processes for handling requests
# - `-b 0.0.0.0:7860`: Binds the server to port 7860 on all network interfaces
# - `app:app`: Runs the Flask app (assuming `app.py` contains the Flask application)
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:7860", "app:learn_api"]
```

Overwriting backend_files/Dockerfile

✓ Setting up a Hugging Face Docker Space for the Backend

```
# Import the login function from the huggingface_hub library
from huggingface_hub import login

# Login to your Hugging Face account using your access token
# Replace "YOUR_HUGGINGFACE_TOKEN" with your actual token
#login(token="YOUR_HUGGINGFACE_TOKEN") # You can get your token from
login(token="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") #Complete the code

# Import the create_repo function from the huggingface_hub library
from huggingface_hub import create_repo
```

✓ Uploading Files to Hugging Face Space (Docker Space)

Double-click (or enter) to edit

```
# for hugging face space authentication to upload files

access_key = "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" #Complete the code to
repo_id = "nkrishnappa/objectPredictionBackend" #Complete the code to

# Login to Hugging Face platform with the access token
login(token=access_key)

# Initialize the API
api = HfApi()

# Upload Streamlit app files stored in the folder called deployment_files
api.upload_folder(
    folder_path="backend_files", # Local folder path in azureml
    repo_id=repo_id, # Hugging face space id
    repo_type="space", # Hugging face repo type "space"
)
```

```
Uploading...: 100% 196M/196M [00:01<00:00, 184MB/s]
CommitInfo(commit_url='https://huggingface.co/spaces/nkrishnappa/object
commit_message='Upload folder using huggingface_hub',
commit_description='', oid='81aa7e955530576b7287575b8901b74ce3fa3e2d',
pr_url=None,
```

✓ **Deployment to frontend**

✓ Streamlit for Interactive UI

```
# Create a folder for storing the files needed for frontend UI deployment
os.makedirs("frontend_files", exist_ok=True)
```

```
%%writefile frontend_files/app.py

import streamlit as st
import requests
import base64 # Import base64

st.title("Object Name Prediction System")

# Input fields
image = st.file_uploader("Upload Image", type=["jpg", "jpeg", "png"])

image_data = {}
if image is not None:
    # Read image as bytes and encode to base64
    image_bytes = image.read()
    encoded_image = base64.b64encode(image_bytes).decode('utf-8')
    image_data = {
        "image_file": encoded_image
    }

# NOTE: `frontend_backend_api_url` is defined in the previous cell. E
# Use a placeholder for now and manually replace if needed, or define
# For proper execution flow in a notebook, ensure `frontend_backend_a
# I will use a direct string for now, assuming the URL from the previ

# The URL was `https://nkrishnappa-objectprediction.hf.space/v1/object
api_url = "https://nkrishnappa-objectprediction.hf.space/v1/objectpre

if st.button("Predict", type='primary'):
    if image_data:
        response = requests.post(api_url, json=image_data)
        if response.status_code == 200:
            result = response.json()
            predicted_value = result["Predicted_Object"]
            st.write("Predicted :", predicted_value)
        else:
            st.error(f"Error in API request: {response.status_code} -
    else:
        st.warning("Please upload an image before predicting.")
```

Overwriting frontend_files/app.py

✓ Dependencies File

```
%%writefile frontend_files/requirements.txt
requests==2.32.3
streamlit==1.45.0
```

Overwriting frontend_files/requirements.txt

✓ Docker File

```
%%writefile frontend_files/Dockerfile
# Use a minimal base image with Python 3.9 installed
FROM python:3.9-slim

# Set the working directory inside the container to /app
WORKDIR /app

# Copy all files from the current directory on the host to the container
COPY . .

# Install Python dependencies listed in requirements.txt
RUN pip3 install -r requirements.txt

# Define the command to run the Streamlit app on port 8501 and make it
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]

# NOTE: Disable XSRF protection for easier external access in order to
```

Overwriting frontend_files/Dockerfile

✓ Uploading Files to Hugging Face Space (Streamlit Space)

```

access_key = "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" #Complete the co
repo_id = "nkrishnappa/objectPredictionFrontend" #Complete the code

# Login to Hugging Face platform with the access token
login(token=access_key)

# Initialize the API
api = HfApi()

# Upload Streamlit app files stored in the folder called deployment_f:
api.upload_folder(
    folder_path="frontend_files", # Local folder path in azureml
    repo_id=repo_id, # Hugging face space id
    repo_type="space", # Hugging face repo type "space"
)

```

```

No files have been modified since last commit. Skipping to prevent empty
WARNING:huggingface_hub.hf_api:No files have been modified since last c
CommitInfo(commit_url='https://huggingface.co/spaces/nkrishnappa/object
commit_message='Upload folder using huggingface_hub',
commit_description='', oid='9eb9461c64dea70e94f87d49d13a5a459eb30b4f',
pr url=None,

```